

Forecasting Automation Pipeline: n8n and Large Language Model-Based Demand Forecasting Automation for E-Commerce Systems

Hasan Safa Aytas
Computer Engineering
Kastamonu University
Kastamonu, Türkiye
hasansafaaytas@gmail.com
0009-0004-7530-3476

Abstract— *This study predicts the sales volume for the upcoming week using available product sales data through the integration of the n8n automation platform and Google Gemini (Large Language Model). The developed system includes a feedback loop that calculates the accuracy rate of past forecasts using the Mean Absolute Percentage Error (MAPE) metric and presents this result to the language model as a verbal prompt. In the project, artificial intelligence corrects its own forecasting bias and autonomously triggers supply chain actions at critical stock levels, alerting the user via email.*

Keywords— *artificial intelligence automation, e-commerce, large language model, demand forecasting, feedback loop, n8n, Gemini, forecasting, automation, pipeline*

I. INTRODUCTION

In today's e-commerce ecosystem the primary key to profitability is the correct management of the supply chain. Excess stock increases storage costs while insufficient stock directly leads to revenue loss and customer dissatisfaction. The critical role of non-linear fluctuations, such as public holidays, weekend peaks, and seasonal campaigns contained within e-commerce sales data on demand forecasting is clearly demonstrated in the M5 Accuracy Competition results [1]. However traditional statistical time series methods remain inadequate in grasping such semantic and calendar contexts and in modeling sudden structural breaks. The application of Large Language Models (LLM) pattern recognition capabilities to numerical data has spawned a new paradigm in the field of Zero-Shot forecasting. It has been scientifically observed that language models can read numerical time series as textual tokens and produce successful forecasts [2] and that numerical inputs can be directly transformed into a "sentence-to-sentence" [3] learning structure. In contrast positioning large language models alone cannot control the model's forecasting bias and error accumulation over time. In order for artificial intelligence agents to evaluate their own decisions the Reflexion (verbal reinforcement learning) framework proposed in the literature [4] argues that models should keep textual feedback (verbal reinforcement) regarding their past errors in memory to improve

their decision processes in subsequent steps and autonomously increase their performance. Because it provides comparability across e-commerce products of different scales in measuring the success of demand forecasts, the Mean Absolute Percentage Error (MAPE) metric is a widely used evaluation criterion in the literature [5]. Based on these theoretical foundations in the literature this study proposes an autonomous demand forecasting pipeline architecture running on the low-code integration tool n8n and the Google Gemini infrastructure. In each forecasting cycle the system compares past forecasts with historical data, calculates MAPE and dynamically injects the achieved success rate into the prompt. Model self-corrects its under-forecast or over-forecast tendencies and autonomously triggers supply chain actions (Gmail API) in critical stock situations.

II. SYSTEM ARCHITECTURE

A. Data Integration and Preparation Layer

The system is periodically triggered via a Schedule Trigger node on n8n. When the flow starts two different datasets are simultaneously pulled from Google Sheets tables which serve as the data source:

- 1) Daily raw sales and stock data.
- 2) Past forecast reports generated by the system.

Missing (null) values in the data cause models to hallucinate or extract incorrect patterns. To overcome this problem a custom JavaScript code node (Code Node) was added to the data pipeline standardizing missing numerical inputs as "0" and missing textual inputs as "Unknown Product" thereby making data cleaning autonomous.

B. Self-Evaluation and MAPE Calculation Layer

This layer is the evaluation layer where errors from past cycles in the system are mathematically measured. To normalize volumetric scale differences between e-commerce products (e.g., a keyboard selling 10 units a day versus headphones selling 150 units), the Mean Absolute Percentage Error (MAPE)

metric recommended by Hyndman and Koehler [5], was used. The algorithm compares the previous week's forecast (F_t) with the total actual sales of the last 7 days (A_t) for each product using the following equation:

$$MAPE = \frac{1}{n} \sum_{t=1}^n \left| \frac{A_t - F_t}{A_t} \right| \times 100 \quad (1)$$

Based on the result obtained from Equation (1) the system determines whether the forecast has an "Over-forecast" or "Under-forecast" tendency and generates a status summary.

C. LLM Output and Closed-Loop Feedback Layer

The calculated performance scorecard and daily data are transmitted to the Google Gemini model. Through the prompt used by the model it is instructed that it is a professional e-commerce data analytics artificial intelligence and it receives the MAPE score between the forecasted values and the actual values as a critique. The model is asked to blend future unstructured event signals such as e-commerce campaigns and holidays. This event reasoning improves the accuracy of numerical forecasting.

D. Reporting Layer

Outputs generated by the LLM are parsed with JavaScript code and saved to the database using Google Sheets.

E. Autonomous Action and Warning Layer

JSON output (the forecast for the next 7 days) is compared with the current stock and autonomous warnings (Gmail) are transmitted to purchasing managers for "Critical" products completing the autonomous chain.

III. EXPERIMENTAL IMPLEMENTATION AND FINDINGS

In order to examine the system's performance and accuracy, 60 days of e-commerce sales and inventory data between March 18 and May 17 2026 belonging to three different product groups (Smart Watch V2, Bluetooth Headphones Pro, Mechanical Keyboard) in the consumer electronics category were simulated. In each forecast cycle it correctly isolated the forecast data for the last 7 days and mathematically compared the realized sales with the model forecasts.

Following the experimental tests and the stabilization of the system the total average forecast error (MAPE) obtained across the system was measured as 9.6% upon the integration of the feedback loop into the model. When the results were analyzed, the following findings were reached:

- Mechanical Keyboard: In this category which has a linear and stable sales graph the error margin dropped below 2% and it was observed that the model forecasted successfully.
- Bluetooth Headphones Pro and Smart Watch V2: For dynamic categories experiencing sudden demand fluctuations during specific periods such as Mother's Day potential forecast deviations were autonomously compensated for. This was achieved by feeding past period error scorecards and calendar context parameters into the system.

IV. CONCLUSION AND FUTURE WORK

In this study a demand forecasting and inventory management system operating with n8n and large language models was developed without the need to train classical machine learning models from scratch. The overall average accuracy value of 90.4% indicates that LLM-based approaches performing zero-shot time series forecasting produce predictions with high accuracy. By comparing previous forecasts with actual data, it was ensured that the artificial intelligence could learn from its own generated errors, preventing the error accumulation problem experienced in open-loop systems.

One of the most significant constraints encountered during the project's development and testing process is that the artificial intelligence occasionally makes lowercase letter errors while producing structured JSON output. This problem was solved by tightening the system prompt with stricter rules.

In the future the following development areas are envisioned to scale this project to larger corporate structures:

- Designing product-based batch processing loops instead of sending all data at once to prevent the language model from hitting the character (token) limit when the number of products and data volume increase.
- Migrating the storage layer to relational data warehouses like PostgreSQL or Google BigQuery to avoid hitting Google Sheets API quota limits and to enhance data security.

In conclusion this autonomous architecture built with low-code tools and language models reduces human error in supply chain processes and offers a sustainable solution that streamlines operations for e-commerce businesses unable to budget for high-cost data science investments.

REFERENCES

- [1] S. Makridakis, E. Spiliotis, and V. Assimakopoulos, "The M5 Accuracy competition: Results, findings and conclusions," *International Journal of Forecasting*, 2021.
- [2] N. Gruver, M. Finzi, S. Qiu, and A. G. Wilson, "Large Language Models Are Zero-Shot Time Series Forecasters," in *Advances in Neural Information Processing Systems* (NeurIPS), 2023.
- [3] Xue, H., & Salim, F. D. PromptCast: A New Prompt-Based Learning Paradigm for Time Series Forecasting. *IEEE Transactions on Knowledge and Data Engineering*, 2024.
- [4] N. Shinn, F. Cassano, E. Berman, A. Gopinath, K. R. Narasimhan, and S. Yao, "Reflexion: Language Agents with Verbal Reinforcement Learning," in *Proceedings of the Annual Conference on Neural Information Processing Systems* (NeurIPS), 2023.
- [5] R. J. Hyndman and A. B. Koehler, "Another look at measures of forecast accuracy," *International Journal of Forecasting*, vol. 22, no. 4, pp. 679–688, 2006, doi: 10.1016/j.ijforecast.2006.03.001.
- [6] D. Reyes Fernández De Bulnes and P. Rosati, "ARIMA Forecasting with LLM-Powered Multi-Agent Coordination for Omnichannel Retail KPIs," in *Proc. 12th IEEE Int. Conf. Intelligent Computing and Information Systems (ICICIS)*, Cairo, Egypt, 2025, doi: 10.1109/ICICIS66182.2025.11313215abc.
- [7] C. Hu, Y. Shi, F. Huang, ve M. Jin, "EventCast: Hybrid Demand Forecasting in E-Commerce with LLM-Based Event Knowledge," *arXiv preprint arXiv:2602.07695*, 2026.